ensemble-next – Kurzanleitung

Version 1.1.0 · Stand 30.10.2026 · `~/projekte/ensemble-next`

Was es macht

Du gibst eine Aufgabe und einen Projektordner vor. **Claude Code** plant und setzt die Aufgabe allein um. Danach prüft **Codex** das Ergebnis **blind**, ohne Lesezugriff auf den Chat und nur lesend. Codex sieht dabei die Aufgabe, den Diff und das Protokoll des Abschlusschecks. Grundlage ist die Messreihe aus Paket 3 mit 16 Läufen: Die Qualität war gleich wie mit Codex als Planer, und der Ablauf war etwa dreimal so schnell (Median 57 s statt 156 s).

1 · Start	**2 · Claude arbeitet**	**3 · Abschlusscheck**	**4 · Blinde Prüfung**	**5 · Urteil**
`/collab-next` oder `collab-launch.sh`	plant, schreibt den Code und testet	automatisch beim Stopp: npm-Skripte, pytest	Codex, read-only, ca. 20 s bis wenige Minuten	`review/verdict.txt`; committen musst du selbst

Einmalig: Server starten

In einem **normalen Terminal**, nicht aus der Claude-Desktop-App heraus. Das Fenster bleibt offen:

```
cd ~/projekte/ensemble-next && npm run dev
curl -s localhost:23100/api/v1/health      # → {"status":"healthy","version":"1.1.0"}
```

Voraussetzungen: Node, tmux, python3 sowie die CLIs `claude` und `codex`, beide angemeldet.

Einen Auftrag starten

Der Projektordner muss ein Git-Repo mit mindestens einem Commit sein. Die Prüfung vergleicht gegen diesen Stand. Ein neuer Ordner geht zum Beispiel so:

```
mkdir -p ~/projekte/mein-test && cd ~/projekte/mein-test && git init -q && echo "# Test" > READM
E.md && git add . && git commit -qm Start
```

A · In Claude Code (empfohlen), im Projektordner:

```
/collab-next Schreibe … Alle Tests müssen grü
n sein.
```

Claude startet das Team, zeigt die Nachrichten und am Ende das Urteil.

B · Direkt im Terminal:

```
~/projekte/ensemble-next/scripts/collab-launc
h.sh \
  ~/projekte/mein-test "Aufgabe …"
```

Die Ausgabe nennt die `TEAM_ID` und den Befehl `tmux attach` zum Zuschauen.

Gute Aufträge nennen ein prüfbares Ziel, zum Beispiel „…, mit Tests, alle Tests grün“, „`test/` nicht ändern“ oder „nur Datei X“. An diesen Kriterien misst Codex das Ergebnis.

Ergebnis lesen

Alles liegt unter `/tmp/ensemble-next/<TEAM_ID>/`. Der Ordner wird beim Neustart des Rechners gelöscht.

Datei	Inhalt
`summary.txt`	Zusammenfassung: Rückmeldung von Claude, geänderte Dateien, Tests, Abschlusscheck
`review/verdict.txt`	Urteil von Codex mit jedem Kriterium und Befunden samt Zeilenangabe; die Prüfung ist fertig, sobald `review/.done` existiert
`review/error.txt`	nur wenn die Prüfung kein Urteil liefern konnte

Urteile und was du tust

Urteil	Bedeutung	Nächster Schritt
ACCEPTED	Alle Kriterien sind erfüllt, der Abschlusscheck steht auf PASS, es gibt keinen Blocker.	Diff ansehen, dann selbst committen.
CHANGES_REQUESTED	Mindestens ein Kriterium ist nicht erfüllt oder nicht belegbar, die Lücken sind behebbar.	Befunde lesen, dann nachbessern lassen oder selbst entscheiden.
REJECTED	Das Ergebnis geht in die falsche Richtung, Prüfungen wurden abgeschwächt, oder es wurde weit außerhalb der Aufgabe geändert.	Nicht übernehmen, Auftrag schärfen.

Meldet Claude selbst `BLOCKED` oder `NEEDS_DECISION`, läuft keine Prüfung. Dann bist du gefragt, zum Beispiel weil sich Vorgaben widersprechen.

Abschlusscheck (automatisch)

Der Check läuft jedes Mal, wenn Claude stoppt, und ist der Beleg für „Tests grün“. Ausgeführt wird nur, was das Projekt selbst festlegt:

- **JavaScript/TypeScript:** die npm-Skripte `typecheck`, `lint`, `test` und `build`, sofern sie in `package.json` stehen.
- **Python:** `pytest`, sobald es `test_*.py`, `conftest.py` oder eine pytest-Konfiguration gibt. pytest läuft mit dem `.venv` des Projekts, falls vorhanden.

Status	Bedeutung
PASS	Alle Prüfungen sind grün.
FAIL	Claude wird bis zu dreimal zum Nachbessern zurückgeschickt.
INCOMPLETE	Zeitüberschreitung, oder npm bzw. pytest fehlt.
SKIPPED	Das Projekt legt keine Prüfungen fest. Codex kann „Tests grün“ dann nicht bestätigen.

Schalter (Umgebungsvariablen)

Variable	Wirkung
`COLLAB_AGENTS=codex,claude`	Echtes Team statt Claude allein; der erste Agent übernimmt die Leitung.
`COLLAB_AUTO_REVIEW=0`	Keine blinde Prüfung für diesen Start.
`ENSEMBLE_AUTO_REVIEW=0`	Keine Prüfung für den ganzen Server (beim Serverstart setzen).
`COLLAB_AGENT_WAIT_SECS`	Wartezeit auf die erste Nachricht beim Start (Standard 60 s).
`COLLAB_REVIEW_RETRY_S`	Pause vor einem neuen Versuch, wenn Codex „at capacity“ meldet (Standard 120 s, höchstens 3 Versuche).

Wenn etwas hakt

- **Keine Prüfung, obwohl READY_FOR_REVIEW gemeldet wurde:** Fehlt `review/started`, steht der Grund im Server-Terminal (Zeile „blind review not started“). Liegt `review/error.txt` vor, sieh in `review/codex.log` nach.
- **Agent startet nicht:** `tmux attach -t $(head -1 /tmp/ensemble-next/<TEAM_ID>/sessions)` zeigt, wo er hängt. Den tmux-Server im normalen Terminal starten.
- **Prüfung von Hand wiederholen:** `scripts/collab-review.sh <Team>/review <Projektordner> [<Basis-Commit>]`

Wichtig: Das Team committet nichts. Den Commit machst du selbst, nachdem du das Urteil und den Diff gesehen hast. Jede Prüfung verbraucht etwas Codex-Kontingent. Das bisherige System (`~/ensemble`, `/collab`, Port 23000) ist unverändert und bleibt der Rückfallweg.